

DotHUD

Minimalist Dot-Matrix PC Widget Engine (Servlets, JSP & JDBC Architecture)

Project Title:	DotHUD – Minimalist Dot-Matrix PC Widget Suite
Core Tech Stack:	Java Servlets, JSP (JavaServer Pages), JDBC & MySQL Database
Architecture Pattern:	MVC (Model-View-Controller) with DAO Data Access Pattern
Degree & Year:	BSc IT – 3rd Year (Final Year Project Submission)

1. Executive Summary & Problem Statement

Modern operating systems (Windows and macOS) lack a clean, unified dot-matrix widget ecosystem. Existing tools like Rainmeter are 15-year-old legacy Windows-only programs requiring complex scripting.

DotHUD is a modern, cross-platform PC widget suite built using core Advanced Java technologies: **Java Servlets**, **JSP (JavaServer Pages)**, **JDBC (Java Database Connectivity)**, and **MySQL Database**. It provides users with dot-matrix styled desktop widgets (Clock, System Hardware Telemetry, App Launchers, Sticky Notes, and Live REST API Widgets) using a clean Model-View-Controller (MVC) architecture.

/// DotHUD - CONCEPTUAL UI HUD ///	
■ Dot Clock: 10:42:05 PM	■ Hardware: CPU: 38% RAM: 6.2 GB
■ App Launcher: [VS Code] [Terminal]	■ Live API: BTC: \$64,200 Weather: 24°C

2. Key System Modules

1. Dot-Matrix Clock & Timer Digital/Analog dot-matrix clocks, world timezone indicators, and countdown timer.	2. System Hardware Monitor Real-time CPU %, RAM, battery health, and disk telemetry via Java OSHI library.
3. App & Script Launcher Launch local PC executables (code.exe, cmd.exe) via Java ProcessBuilder.	4. Dynamic REST API Engine Enter any REST API URL + key (Weather, Crypto, GitHub) to auto-render a live widget.
5. Quick Task & Note Board Minimalist dot-matrix sticky note board and task checklist stored via JDBC in MySQL.	6. Drag & Drop Visual Grid Interactive JSP grid customizer to move, resize, and lock dot-matrix widgets visually.

3. MVC Architecture (Servlets, JSP & JDBC)

The project follows the standard college curriculum **MVC (Model-View-Controller)** architecture using pure Servlets, JSP, JDBC, and MySQL:

MVC Layer	Technology / Component	Role & Implementation Details
Controller (Servlets)	WidgetServlet.java TelemetryServlet.java AppLauncherServlet.java	Java Servlets (`@WebServlet`) handling GET/POST requests, session management, JSON serialization, and system commands.
View (JSP / HTML)	index.jsp config.jsp JSTL / CSS Dot-Matrix Theme	JavaServer Pages rendering the dot-matrix widget dashboard, admin config panel, and grid layout dynamically.
Model & DAO (JDBC & MySQL)	DBConnection.java WidgetDAO.java MySQL (`dothud_db`)	JDBC Data Access Layer executing SQL queries with `PreparedStatement` (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) for widget persistence.
System API	Java OSHI Library Java `ProcessBuilder`	Backend Java services measuring CPU/RAM hardware metrics and launching desktop applications.

4. MySQL Database Schema (JDBC)

Stored in MySQL database dothud_db via JDBC prepared statements:

Column Name	Data Type	Description
id	INT PRIMARY KEY AUTO_INCREMENT	Unique widget identifier
title	VARCHAR(100)	Widget display title
widget_type	VARCHAR(50)	Clock, System, Launcher, REST_API, Notes
pos_x, pos_y	INT, INT	Grid coordinates for drag-and-drop
custom_api_url	VARCHAR(255)	Target REST API URL for dynamic widget
target_json_key	VARCHAR(100)	JSON field to extract and display

5. Implementation Phases & Deliverables

- **Phase 1 (Week 1):** Environment setup, MySQL dothud_db schema creation, JDBC DBConnection & WidgetDAO implementation.
- **Phase 2 (Week 2):** Java Servlets development (WidgetServlet, TelemetryServlet using OSHI library).
- **Phase 3 (Week 3):** JSP view creation (index.jsp, config.jsp), CSS dot-matrix styling, drag-and-drop JS grid engine.
- **Phase 4 (Week 4):** Dynamic REST API Proxy Servlet development and local application execution engine (`ProcessBuilder`).
- **Phase 5 (Week 5):** System integration testing, viva presentation deck preparation, and final project documentation.